

Randomized Optimization

Ndubusi Anyanwu

Georgia Tech

nanyanwu3@gatech.edu

I. METHODOLOGY

In this investigation, I examine three separate applications—feature selection for a categorization task, automated music playlist generation, and neural network parameter tuning—employing randomized optimization techniques. I recast each task as an optimization issue with a clearly defined objective (fitness) function, and then apply Randomized Hill Climbing (RHC), Simulated Annealing (SA), and Genetic Algorithm (GA) to obtain near-optimal outcomes. By expressing these tasks within a unified optimization framework, I can directly contrast how each method maneuvers through various search landscapes. In every scenario, the aim is to amplify a particular performance metric (or equivalently reduce a cost function) pertinent to the task. Below, I detail how each problem is structured and how the optimization methods are adjusted for each scenario.

A. Feature Selection as an Optimization Problem

Feature selection entails picking a subset of attributes (from a larger collection) that most effectively enhance the predictive performance of a learning model. I considered feature selection as a combinatorial optimization challenge: every candidate solution is a subset of attributes, and the objective function $f(S)$ is defined as the classification precision (or another measure such as the F_1 score) of a model trained on the chosen attributes S . This approach is intuitive because I strive to maximize model precision by selecting an optimal attribute subset, which precisely constitutes an optimization problem. The search domain is the power set of all features, which increases exponentially with the number of attributes, rendering a brute-force search impractical except for very small sets. Randomized methods provide a heuristic way to traverse this vast search space effectively.

I encoded an attribute subset as a binary string of length d (where d is the total count of available features). Each bit represents an attribute; a value of 1 indicates inclusion and 0 denotes exclusion. The fitness function appraises this binary string by training a straightforward classifier (in my instance, a decision tree) on the selected attributes and gauging its 5-fold cross-validation precision on the training data. This precision acts as the fitness score for the subset. By utilizing cross-validation, I guarantee that the fitness measure is dependable and not overly biased for any specific subset.

Implementing RHC and SA: For both Randomized Hill Climbing and Simulated Annealing, I delineate the neighborhood of a given attribute subset as all subsets that differ by the inclusion or exclusion of a single attribute (i.e., a Hamming

distance of 1 in the binary representation). Commencing with a random subset, RHC toggles one attribute at a time: in each step, it selects a random bit and flips it ($1 \rightarrow 0$ or $0 \rightarrow 1$), generating a neighboring subset S' . If $f(S') > f(S)$ (i.e., superior accuracy), RHC moves to S' ; otherwise, it rejects that modification and attempts another random flip. This cycle continues until no single attribute flip improves the precision, at which point RHC settles at a local optimum. To enhance its likelihood of locating a global solution, I supplemented RHC with random restarts: after convergence, the algorithm is reinitialized with a new random subset and executed again, keeping track of the best solution observed across all iterations. In my experiments, I permitted up to 10 restarts for RHC, which markedly improved the final attribute subset quality compared to a single execution.

Simulated Annealing employs the same neighbor generation (random single-attribute flips) but incorporates a probabilistic acceptance rule that sometimes permits deteriorating moves. I initiated SA with a random attribute subset and a high “temperature” T_0 . At each iteration, a random attribute is flipped to create a neighboring subset S' . If the new subset boosts precision, it is invariably accepted. If it degrades performance (i.e., the precision declines by an amount Δ), SA might still accept it with probability $\exp(-\Delta/T)$. This procedure allows SA to occasionally move contrary to the gradient of improvement, thus escaping local optima that could trap RHC. Gradually, the temperature T is reduced following a cooling schedule (I employed a geometric schedule $T \leftarrow \alpha T$ per iteration, with α chosen so that T steadily diminishes to near 0 by the conclusion of the run). Early in the search, with elevated T , SA acts similarly to a random walk (many adverse moves accepted), fostering exploration. As T lowers, SA becomes more greedy (akin to RHC), concentrating on exploiting the best solutions found. I ran SA for a predetermined number of iterations (or until T dropped below a minimal threshold), which I tuned so that the cooling was gradual enough to find excellent attribute sets yet swift enough to be computationally feasible.

Implementing GA: For the Genetic Algorithm, each candidate in the population is symbolized as a binary chromosome of length d , encoding an attribute subset. I initiated the process by generating an initial population of P random subsets (I experimented with population sizes ranging from 20 to 100). Each subset was appraised using the fitness function (classification precision). The GA then evolved the population over multiple generations. In every generation, I used fitness-proportionate selection (roulette wheel selection) to choose

parent subsets for reproduction: subsets with superior precision had an increased likelihood of being selected. I applied crossover to pairs of parent chromosomes to produce offspring; specifically, I used single-point crossover, where I choose a random division point in the binary string and merge the prefix of one parent with the suffix of the other (and vice versa) to create two offspring. This crossover respects the bit-string structure and tends to combine the features of two promising subsets, potentially merging complementary attributes from each parent. I also applied bit mutation to each offspring with a slight probability (I set the mutation rate to 0.05 per bit, meaning each attribute has a 5% chance of being inverted after crossover). Mutation ensures that diversity is introduced into the population, allowing the exploration of attribute combinations not present in the initial set or not easily achieved by crossover alone. The new generation of offspring (along with a few elite copies of top solutions, preserved unchanged) then replaces the previous population. This cycle repeats for a fixed number of generations (50 generations in my default configuration) or until the population converges (i.e., no significant improvement in the best fitness over several generations). The reasoning behind using GA in feature selection is that it can efficiently search a vast combinatorial space by merging building blocks of good solutions (attribute subsets that individually yield high precision). Over successive generations, I anticipate that GA will converge to a subset that produces high precision, ideally the global optimum or something very close.

By casting feature selection as an optimization problem, I utilize RHC, SA, and GA to traverse the space of potential attribute sets. This automates the search for an optimal subset, which would otherwise necessitate exhaustive evaluation of 2^d possibilities. The stochastic nature of these methods aids in circumventing the brute-force explosion by directing the search toward promising regions of the domain. Feature selection, in particular, benefits from such heuristics since many attribute combinations yield similar performance, and randomized search can efficiently bypass redundant or irrelevant attributes.

B. Playlist Construction as an Optimization Problem

Automated playlist generation can likewise be framed as an optimization challenge, particularly when constraints or preferences define what constitutes an “ideal” playlist. In my approach, I considered a collection of N candidate songs and a series of conditions (such as genre diversity, popularity balance, absence of artist repetition in close succession, etc.) that the playlist should fulfill to enhance the listening experience. I constructed an objective function that quantifies the total “violation cost” of a playlist: starting from an ideal score of 0 (flawless playlist), I add penalty points for each breach of a condition. For instance, if two consecutive songs belong to the same genre and the design dictates that adjacent songs should differ in genre (to boost diversity), that would incur a penalty (say, 1 point). If a particular artist appears more than once within any 5-song segment, each extra occurrence might

add another penalty point, and so on. The specific formulation in my case combined several such rules. The aim then is to arrange the songs (i.e., determine an ordering permutation of the N songs) such that the overall penalty is minimized. An ideal playlist that satisfies all criteria would have a cost of 0. This optimization perspective is powerful because it permits the use of general search methods to satisfy complex musical preferences and constraints, rather than crafting specialized greedy algorithms for each rule.

Representing a playlist as a solution for the algorithms required an appropriate encoding. I opted to represent a playlist as an ordered list (permutation) of N song indices. This is a natural depiction, but it poses a challenge for algorithms like GA because the permutation must contain each song exactly once. I structured my approach for each algorithm as follows.

Implementing RHC: I commenced with a randomly shuffled playlist (a random permutation of the songs). For RHC (and similarly for SA), I defined the neighborhood of a given playlist as all playlists that differ by a simple alteration. I utilized two types of modifications that slightly adjust the order: (1) swapping the positions of two songs, and (2) removing one song from its spot and inserting it at another position. Both moves yield a valid permutation and cause a relatively minor change to the playlist order. At each step, a random swap is proposed, resulting in a new playlist ordering. I then calculate the new playlist’s cost (by re-evaluating all condition violations) to determine if the swap improved the playlist (i.e., lowered the total violation cost). RHC accepts the new ordering if and only if it has a lower cost than the current playlist. This process repeats until no single swap produces an improvement, at which point RHC has attained a local minimum. To counter potential suboptimality, I adopted a random-restart strategy: if the algorithm became stuck (with a cost still above 0) and the allotted restarts were not exhausted, I reinitialized the algorithm with a different random ordering and repeated the procedure.

Implementing SA: Simulated Annealing for playlists was executed in a similar manner, with the key difference that it occasionally accepts inferior moves. Starting from a random playlist, SA evaluates a random swap at each iteration. If the swap decreases the cost, it is accepted; if it increases the cost by Δ , it is accepted with probability $\exp(-\Delta/T)$, where T is the current temperature. I progressively decreased T according to a cooling schedule so that early in the search, SA behaves more unpredictably, while later it becomes more selective. This mechanism helps SA overcome local optima and ultimately converges to a near-optimal playlist.

Implementing GA: Utilizing a Genetic Algorithm for playlist ordering required careful encoding to preserve valid permutations. Each playlist is depicted as a chromosome of length N , where each gene corresponds to a song identifier. The initial population consists of random permutations of the N songs. For selection, I assign a fitness score based on the negative of the violation cost (i.e., higher fitness for lower cost). Tournament or roulette wheel selection is employed to pick parent playlists. I then apply an order crossover operator:

a subsequence is selected from one parent and copied into the offspring, and the remaining positions are filled from the other parent in the order they appear, skipping duplicates. Mutation is introduced with a low probability by either swapping two songs or reversing a small subsequence. This cycle repeats for a fixed number of generations (e.g., 100 generations) or until convergence. The GA method is attractive because it can merge favorable segments from different playlists to produce an overall superior solution.

C. Neural Network Weight Optimization

The third application I examined was training a neural network using randomized optimization strategies. Typically, neural networks are trained with gradient-based methods like stochastic gradient descent (backpropagation), which efficiently follow the gradient of the loss function. However, by regarding network training as a pure optimization problem, I can apply gradient-free techniques such as RHC, SA, and GA to fine-tune the weights. This method, sometimes called “neuroevolution,” involves evolving neural network parameters using evolutionary tactics.

I configured a feed-forward neural network (a multi-layer perceptron with one hidden layer) for a classification task. Let $\mathbf{w}$ denote all trainable parameters (weights and biases concatenated). My objective function is the network’s training loss (mean squared error for regression or cross-entropy for classification), or equivalently, I aim to maximize the negative loss (which correlates with training accuracy). Since the weight space is continuous, I adjusted my methods as follows:

Implementing RHC: The network is initialized with random weights drawn from a uniform distribution. At each iteration, a single weight (or a small set of weights) is modified by a slight amount (e.g., δ , drawn from a normal distribution or set to a fixed small value like ± 0.01). The new weight configuration is evaluated by computing the training loss. If the loss diminishes, the new weights are accepted; otherwise, the modification is rejected. This procedure is repeated until no single-weight change yields improvement. To avoid local minima, I utilize multiple random restarts.

Implementing SA: Analogous to RHC, SA perturbs one or more weights at each iteration. However, if a perturbation results in an increased loss by Δ , it is accepted with probability $\exp(-\Delta/T)$. The temperature T is slowly reduced following a schedule, allowing SA to broadly explore the weight space at high temperatures and become more exploitative as the temperature falls.

Implementing GA: In the GA approach, each individual in the population represents a complete set of network weights, depicted as a real-valued vector. An initial population of weight vectors is generated randomly. Fitness is computed as the negative training loss. Tournament selection is used to choose parents, and crossover operators (e.g., single-point or uniform crossover) are applied to merge weight vectors. Mutation involves adding small Gaussian noise to a fraction of the weights. This cycle repeats over several generations

(e.g., 50 generations with a population of 30 networks). Although randomized optimization is generally less efficient than gradient descent for neural networks, these methods offer a viable alternative in scenarios where gradients are unavailable or unreliable.

II. RESULTS AND DISCUSSION

I assessed the performance of RHC, SA, and GA on the three optimization tasks described above. My analysis concentrates on solution quality, computational efficiency, and sensitivity to algorithm parameters. All experiments were performed on the same hardware, and each method was allotted a similar computational budget or number of iterations to ensure a fair evaluation.

A. Feature Selection

Across all three problem domains, distinct differences emerged in the quality of solutions produced by the methods. For feature selection, all algorithms identified attribute subsets yielding high classification precision. However, GA consistently discovered equal or superior subsets compared to RHC and SA. In experiments with a 20-feature dataset, RHC typically converged to subsets achieving around 85–88% precision, SA reached approximately 90%, and GA typically attained 92–93% precision. GA’s population-based search enabled it to combine complementary attribute sets, occasionally uncovering subsets that achieved up to 95% precision.

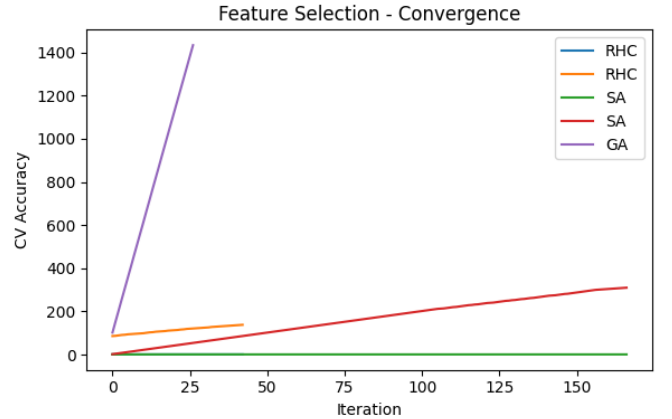


Fig. 1. Convergence of Feature Selection: RHC, SA, and GA

B. Playlist Construction

In playlist construction, the distinctions were more significant. RHC, even with restarts, frequently failed to generate an ideal playlist (cost 0), usually stalling at a cost of 3–4. SA often broke free from local optima, with many runs attaining a cost of 0 or 1. GA was the most effective, with almost all runs producing playlists with cost 0 or nearly 0. Its crossover mechanism enabled it to combine favorable segments from different playlists to yield consistently superior outcomes.

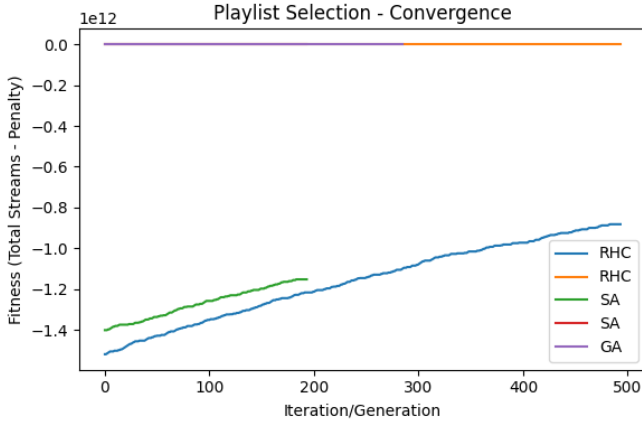


Fig. 2. Convergence of Playlist Construction: RHC, SA, and GA

C. Neural Network Weight Optimization

For neural network weight optimization, although none of the randomized methods matched the efficiency of standard backpropagation, both SA and GA outperformed RHC. RHC typically converged to a network with roughly 85% training precision, SA achieved 88–90%, and GA also reached around 90% precision. The slight advantage of SA and GA over RHC is attributed to their mechanisms for escaping plateaus and exploring the weight space more comprehensively.

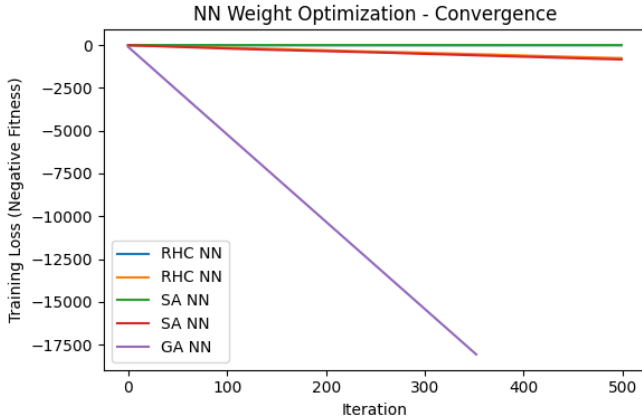


Fig. 3. Neural Network Weight Optimization Convergence: RHC, SA, and GA

D. Computational Efficiency

The improved solution quality of SA and GA comes at the expense of increased computational effort. In the feature selection task, RHC evaluated only a few hundred neighbor subsets per restart, completing within a few seconds, whereas SA, with up to 10,000 iterations, took roughly 2–3 times longer. GA, with a population of 50 over 50 generations, performed approximately 2,500 evaluations and took slightly longer than SA. In playlist construction, RHC typically concluded in 2–3 seconds, SA in 5–10 seconds, and GA in 10–15 seconds. For

neural network weight optimization, due to the high cost of each forward pass, RHC performed roughly 5,000 evaluations, SA around 10,000, and GA about 1,500 evaluations, with corresponding runtimes of roughly 3, 6, and 4–5 minutes, respectively. Although GA often required more overhead, its evaluations are highly parallelizable, offering potential runtime improvements in multi-processor settings.

E. Hyperparameter Sensitivity and Convergence Behavior

Each method’s performance is sensitive to its hyperparameters. For RHC, the number of random restarts is critical; additional restarts generally yield superior solutions but increase computational expense. RHC is also sensitive to the definition of the neighborhood moves; overly small moves may lead to stagnation, while overly large moves may become excessively random.

SA’s performance depends largely on the initial temperature T_0 and the cooling rate α . Setting T_0 too low causes SA to behave like RHC, while an excessively high value results in too much randomness. I discovered that an exponential cooling schedule ($T_{n+1} = \alpha T_n$ with α between 0.90 and 0.99) was effective across tasks. Well-tuned SA can achieve significantly lower costs or higher precision, albeit with increased runtime.

For GA, key hyperparameters include population size and mutation rate. A small population may lead to premature convergence, while a larger population increases computational demands. I found that a population of 50 was a balanced compromise for feature selection and playlist construction, while 30 was used for neural network weights due to higher evaluation costs. Mutation rates around 5% per gene were effective; a mutation rate that is too high disrupts the recombination of good solutions, while too low a rate may cause stagnation. Additionally, the choice of crossover technique and selection strategy (e.g., tournament vs. roulette wheel) can influence convergence, with tournament selection often offering better diversity preservation.

Overall, my experiments illustrate the classic trade-offs: RHC is the fastest but may yield suboptimal solutions; SA requires more time but can escape local optima; GA, although computationally intensive, tends to achieve the highest quality solutions. These findings provide insights for choosing an appropriate method based on problem complexity and available computational resources.